Computer vision, pattern recognition and image retrieval

Laboratory 4

Topic: *Filtration of digital images*

Teacher: Joanna Kulawik, PhD

One of the most important areas of image processing is digital image filtering. These techniques make it possible to extract information contained in an image, improve the visual quality of poor-quality images, and also obtain interesting visual effects.

In this lab, you will learn about some of the most basic and well-known filtering methods. To better illustrate the problem of filtering poor-quality images, you will first create two versions of noisy images. Then, you will filter three images using a selected filter, which will allow you to compare the resulting output images.

To this end, you will create a simple application for performing sample image filtering operations. The application will be developed in MATLAB using a graphical user interface (GUI). Open MATLAB and complete the exercises step by step by following the instructions below. Technical support for MATLAB is available here: http://www.mathworks.com/

In Laboratory 04, your task will be to create a new application 'Lab04.mlapp' using a graphical user interface. Similarly to Laboratory 02, your application will be used for processing digital images. The subsequent tasks will involve adding further functionalities to this application..

Exercise 1

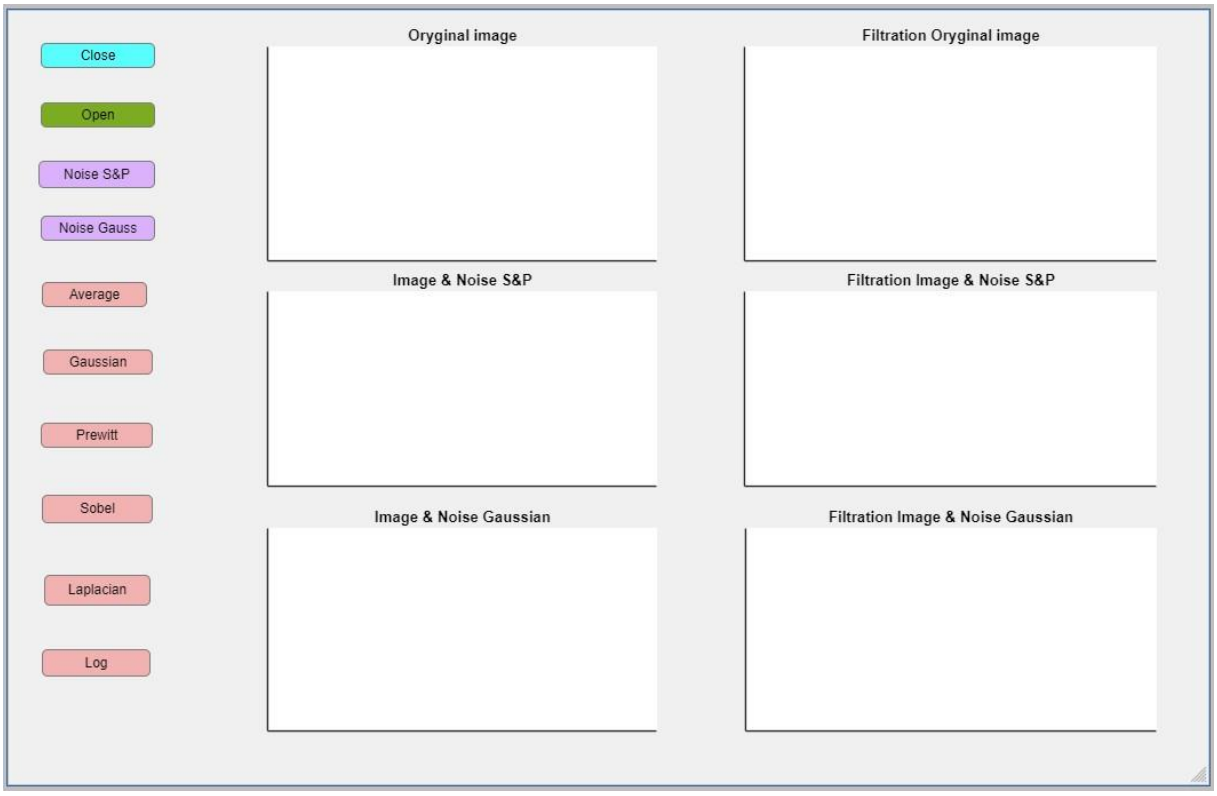
Create a new application 'Lab04.mlapp. ' The form should contain 10 'Button' objects and 6 'Axes' objects.

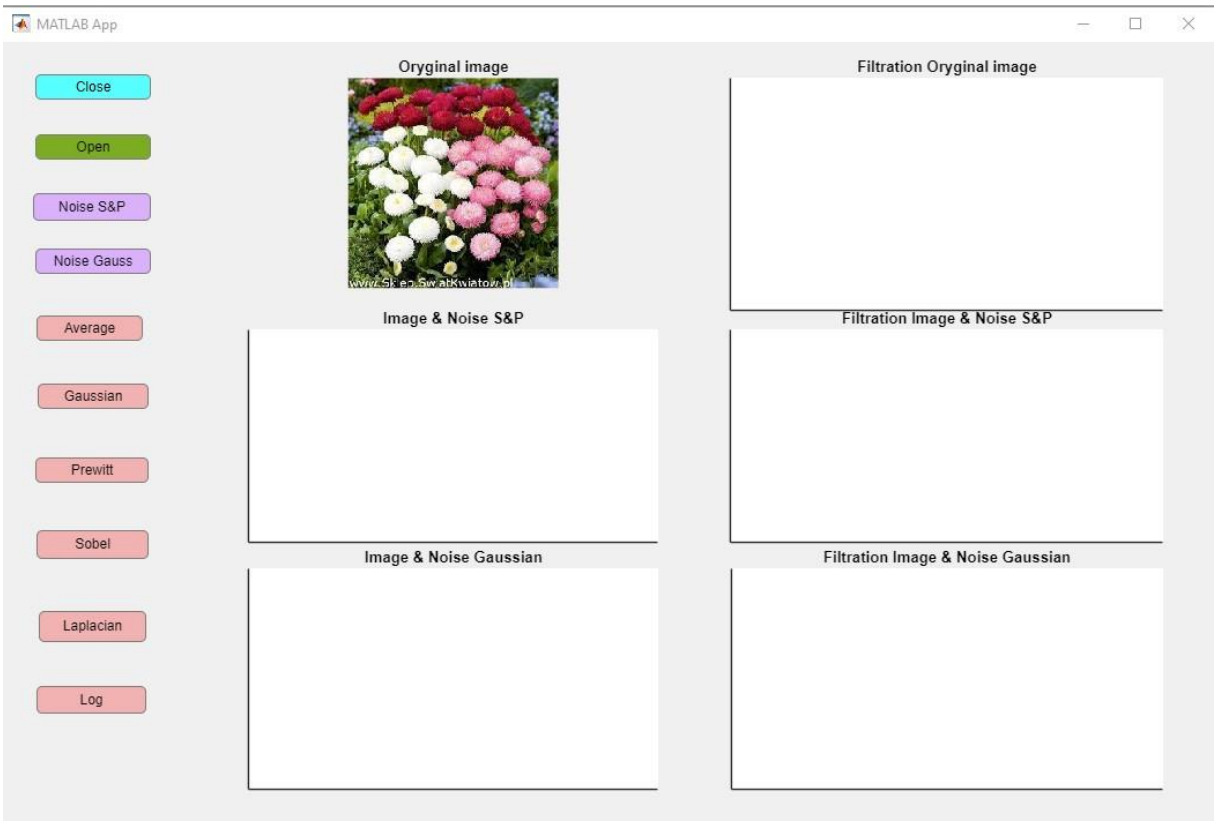
- The first button, 'Close', should close the form.
- The second button, 'Open', should read a color image (in .jpg, .png format) from a selected file and display it in one of the Axes objects.

Detailed instructions on how you can do this can be found in Lab02.

The figures illustrates the expected appearance of the application.

© Joanna Kulawik





© Joanna Kulawik

Exercise 2

For the third button, "Noise S&P", program the application so that the loaded image is corrupted with "salt-and-pepper" noise. To do this, create a copy of the original image and apply the noise to the copy. The noisy image should be a global image so that it can be accessed in other functions (buttons), and it should be displayed in Axes2.

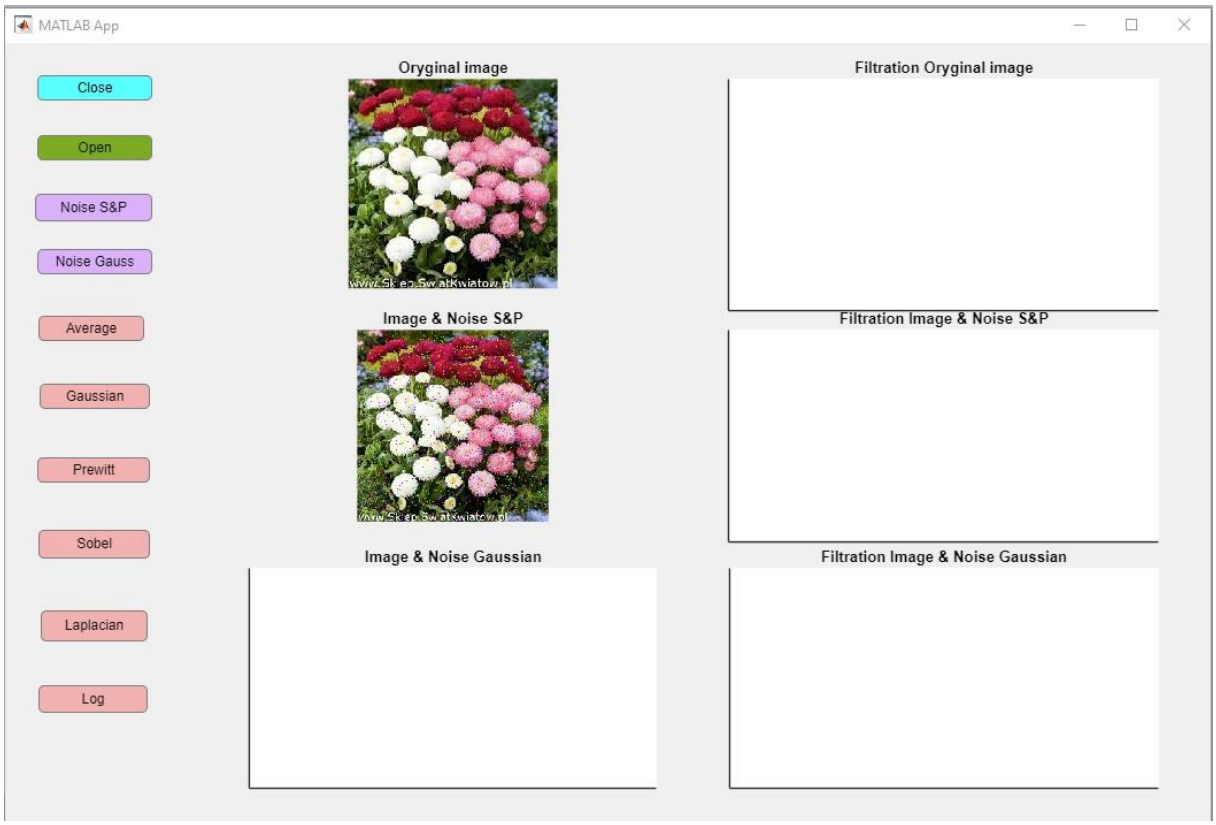
This functionality can be implemented, for example, as follows:

```
global image image_noiseSP
image_noiseSP=imnoise(image, 'salt & pepper');
imshow(image_noiseSP,'Parent',app.UIAxes2);
```

Required commands:

imnoise - introducing noise in the image

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 3

I For the ourth button, "Noise Gauss", program the application so that the loaded image is corrupted with "gaussian" noise. To do this, create a copy of the original image and apply the noise to the copy. The noisy image should be a global image so that it can be accessed in other functions (buttons), and it should be displayed in Axes3.

This functionality can be implemented in a similar way as in Exercise 2.

Required commands:

imnoise - introducing noise in the image

The figures illustrate the expected appearance of the application.

© Joanna Kulawik

Exercise 4

For the fifth button, "Average", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an averaging filter.

As an example:

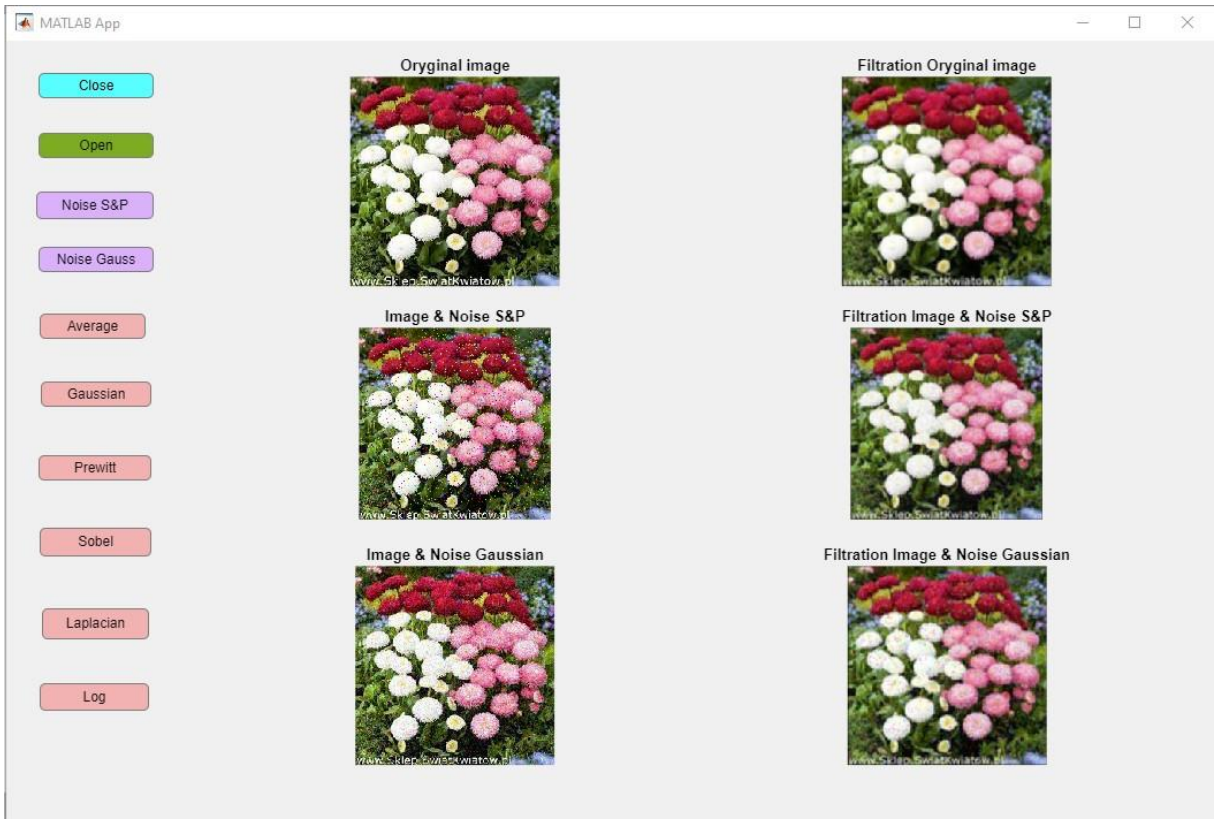
```
f=fspecial("average");
image_average=imfilter(image, f);
```

- Axes4 should display the original image after "averaging" filtration.
- Axes5 should display the noisy "salt & pepper" image after "averaging" filtration.
- Axes6 should display the noisy "Gaussian" image after "averaging" filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 5

For the fifth button, " Gaussian ", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an "gaussian" filter.

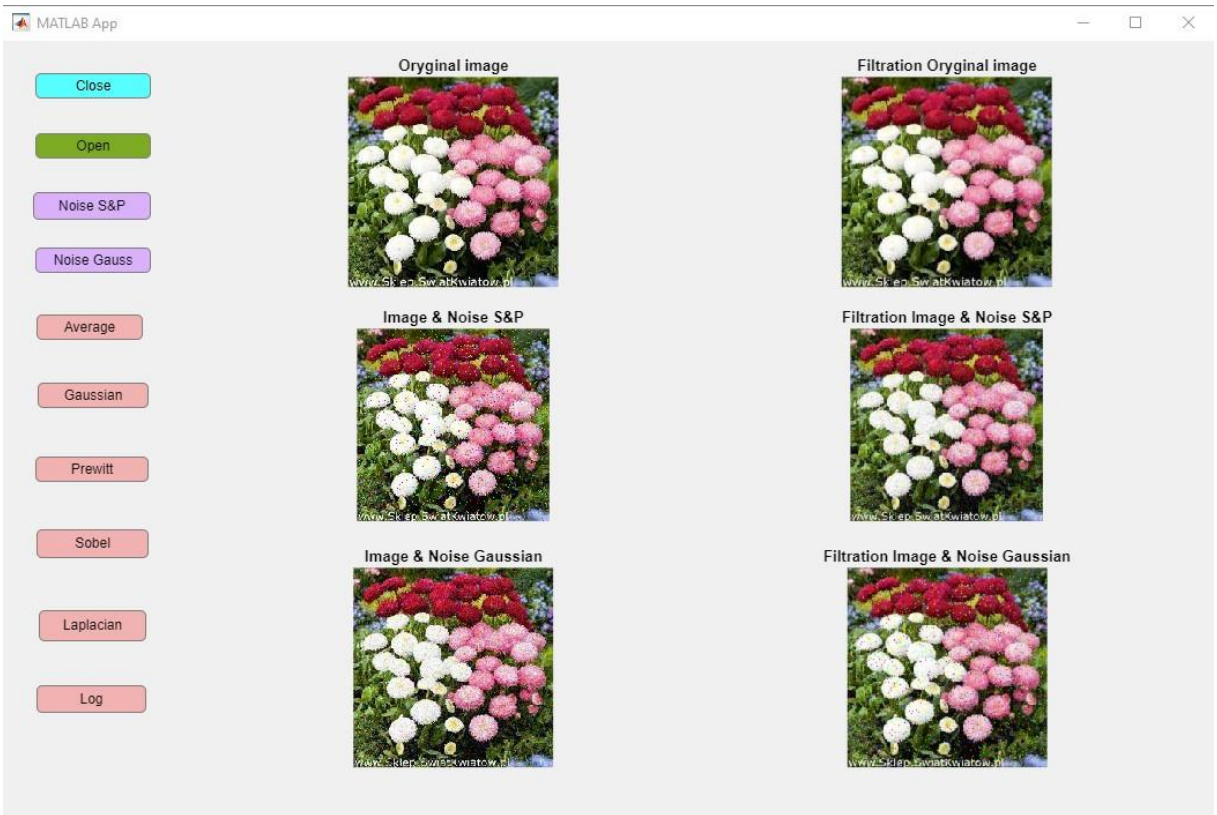
This functionality can be implemented in a similar way as in Exercise 4.

- Axes4 should display the original image after "gaussian" filtration.
- Axes5 should display the noisy "salt & pepper" image after "gaussian" filtration.
- Axes6 should display the noisy "Gaussian" image after "gaussian" filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 6

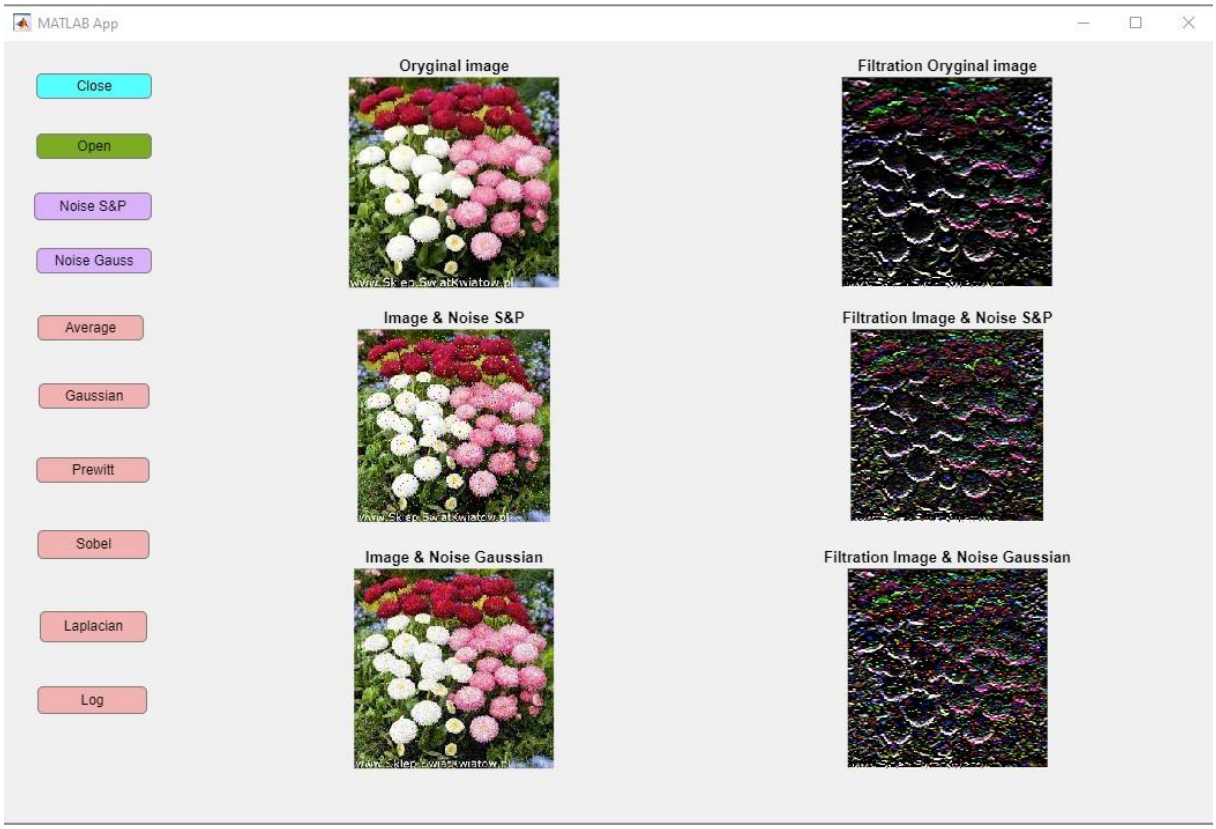
For the next button, " Prewitt ", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an " Prewitt " filter.

- Axes4 should display the original image after " Prewitt " filtration.
- Axes5 should display the noisy "salt & pepper" image after " Prewitt " filtration.
- Axes6 should display the noisy "Gaussian" image after " Prewitt " filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 7

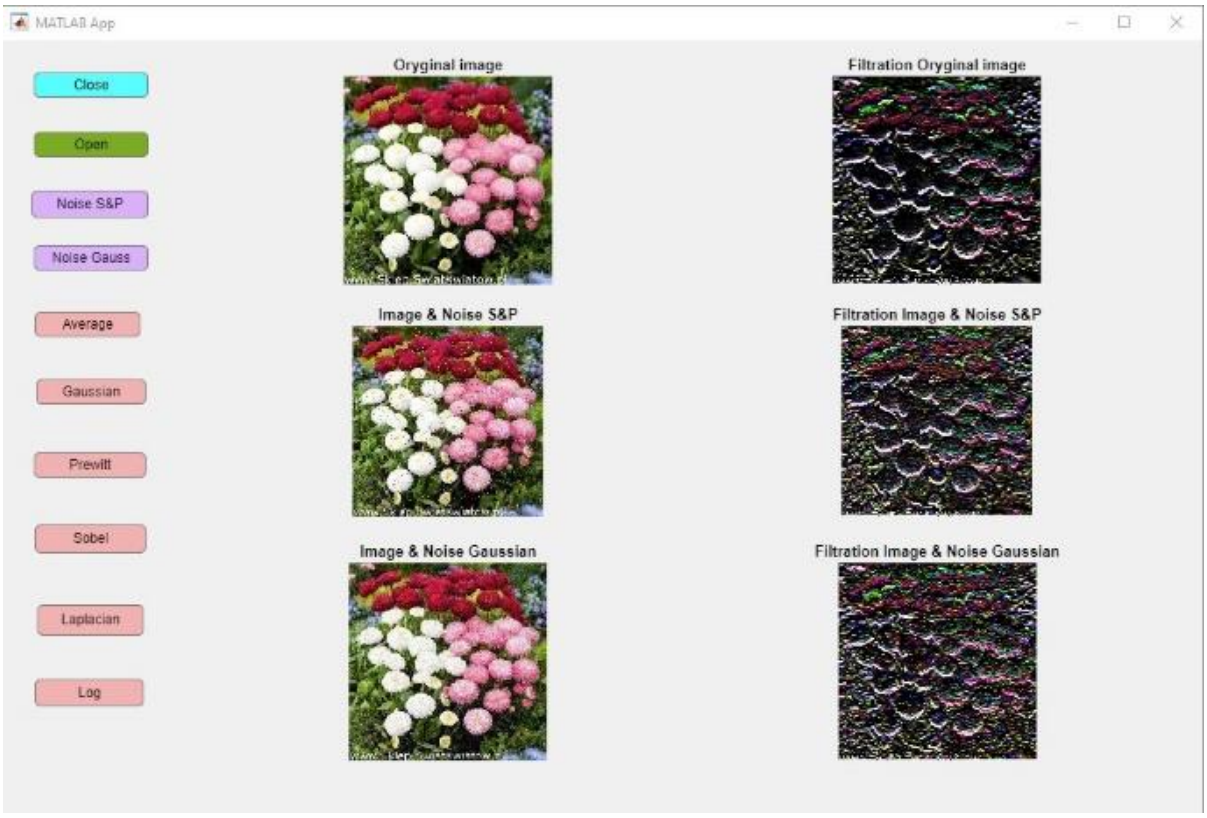
For the next button, " Sobel ", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an "Sobel" filter.

- Axes4 should display the original image after " Sobel " filtration.
- Axes5 should display the noisy "salt & pepper" image after " Sobel " filtration.
- Axes6 should display the noisy "Gaussian" image after " Sobel " filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 8

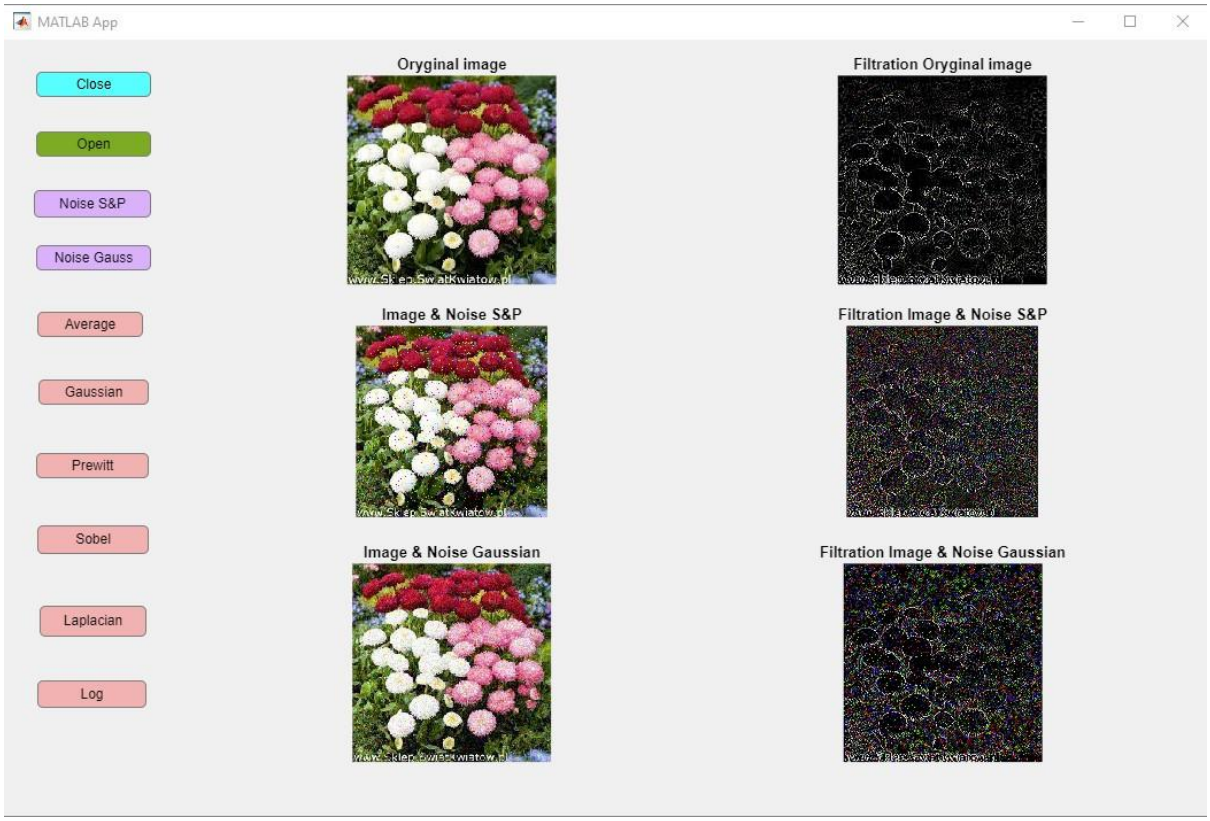
For the next button, "Laplacian", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an "Laplacian" filter.

- Axes4 should display the original image after "Laplacian" filtration.
- Axes5 should display the noisy "salt & pepper" image after "Laplacian" filtration.
- Axes6 should display the noisy "Gaussian" image after "Laplacian" filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



© Joanna Kulawik

Exercise 9

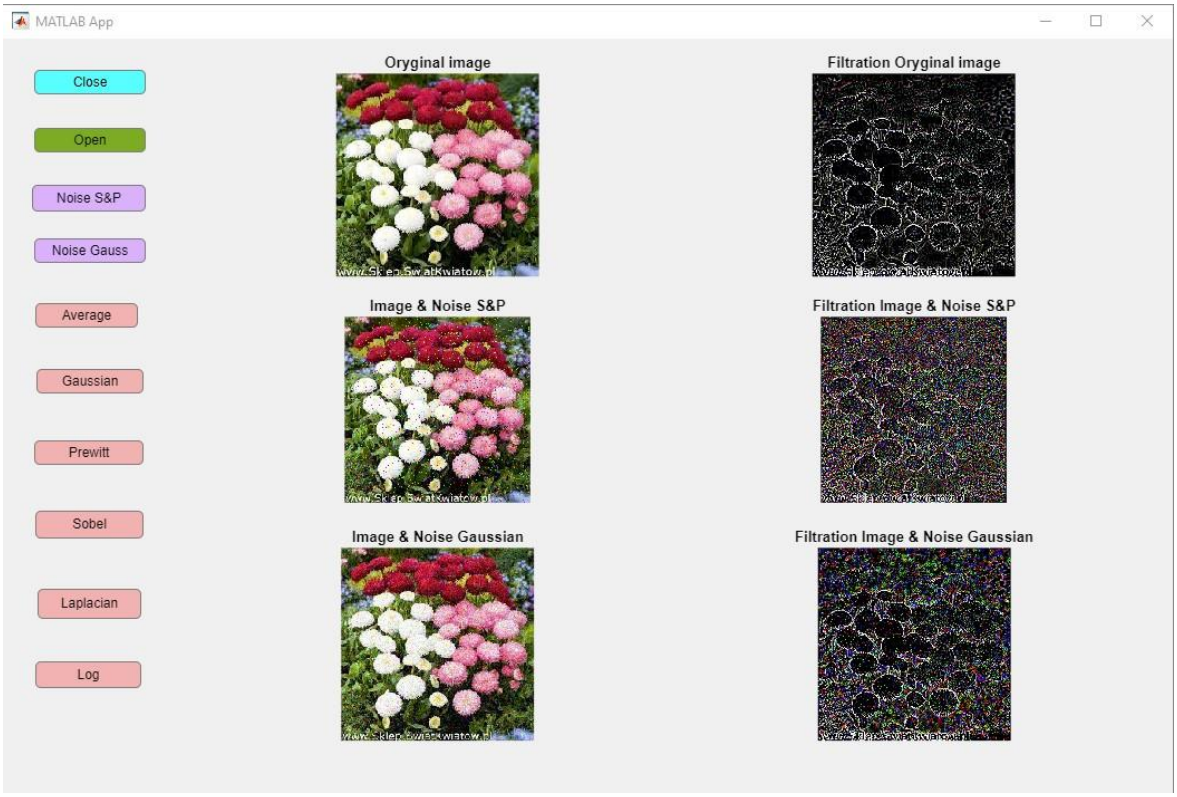
For the next button, "Log", program the filtering operation for three color images: the loaded original image, the noisy "salt & pepper" image, and the noisy "Gaussian" image. Use an "Log" filter.

- Axes4 should display the original image after "Log" filtration.
- Axes5 should display the noisy "salt & pepper" image after "Log" filtration.
- Axes6 should display the noisy "Gaussian" image after "Log" filtration.

Required commands:

fspecial - creates a filter kernel of the selected type (e.g., average, Gaussian, Prewitt, Sobel, Laplacian, LoG, etc.)
imfilter - performs convolution of an image with a filter kernel

The figures illustrate the expected appearance of the application.



Did you notice differences in the results obtained for the high-quality image and the noisy images when applying the individual filters?

Please send only the „Lab04.mlapp" file to the moodle platform as an answer.

© Joanna Kulawik